



Local Language Models for Spoken Rover Task Planning with Clarification Dialogue

by

U. V. S. Rosant

Research assignment presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Electrical and Electronic) in the Faculty of Engineering at Stellenbosch University

Supervisor: Prof. TODO Surname

2026

Someone.

Acknowledgements

* Supervisor X.
* Funding Y.

> *A very long quote*
> *from someone very important.*
> – Mr M

Declaration

By submitting this research assignment electronically, I declare that the entirety of the work contained therein is my own, original work, that I am the sole author thereof (save to the extent explicitly otherwise stated), that reproduction and publication thereof by Stellenbosch University will not infringe any third party rights and that I have not previously in its entirety or in part submitted it for obtaining any qualification.

Date: November 2026

Copyright © 2026 Stellenbosch University
All rights reserved.

Abstract

Uittreksel

Table of Contents

Abstract **iii**

Uittreksel **iv**

Table of contents **v**

List of figures **vi**

List of tables **vi**

Nomenclature **viii**

Acronyms and abbreviations ix

1. Introduction **1**

1.1. Background 1

1.2. Problem Statement and Research Questions 2

1.3. Objectives 3

1.4. Scope and Limitations 4

1.5. Report Outline 5

2. Literature Study **6**

2.1. Language models as robot task planners 6

2.2. Grounding plans in a bounded action set 8

2.3. Clarification and ambiguity 10

2.4. Locally hosted models 12

2.5. Compositional depth and hallucination 13

2.6. Speech recognition for local languages 14

2.7. Research gap 15

3. System Design **16**

3.1. User and functional requirements 16

3.2. System architecture 16

3.2.1. Off-board inference and the thin-client split 16

3.2.2. Two-rate control: waypoint intents versus the reactive layer 19
3.2.3. The end-to-end pipeline 20
3.2.4. Status lights 20
3.3. The capability vocabulary 21
3.4. Clarification loop design 22
3.4.0.1. Where the loop sits 22
3.4.1. The dialogue as a state machine 23
3.4.2. Checking the plan before reading it back 23
3.4.2.1. Confirmation 24
3.4.3. The two later moments 24
3.5. Navigation as instrumentation 25
3.5.1. The line follower as the reactive layer 25
3.6. Summary of design decisions 26

4. Implementation **27**
4.1. Speech-to-text subsystem 27
4.2. Planner 27
4.3. Vision-language verification 27
4.4. Clarification manager 27
4.5. Rover client and reactive layer 27
4.6. Communication protocol and dashboard 27
4.7. Logging and measurement instrumentation 27

5. Experimental Evaluation **28**
5.1. Experimental methodology 28
5.1.1. Command corpus design 28
5.1.2. Evaluation metrics 29
5.2. Plan correctness under vague commands 29
5.3. Executability on the physical chassis 31
5.4. Compositional depth and hallucination onset 32
5.5. Reference comparison against cloud models 34
5.6. Discussion 35

6. Conclusion **36**
6.1. Answers to the research questions 36
6.2. Contributions 36
6.3. Limitations 36
6.4. Recommendations for future work 36

References **37**

A. Appendix title goes here **39**

List of Figures

2.1. Process overview of RAP generation before rover departures 8
2.2. SayCan combining language-model relevance scores with value-function affordance scores to select the next skill, reproduced from Ahn et al. [1]. 9
2.3. The KnowNo pipeline: an LLM planner proposes next steps with confidence scores, conformal prediction narrows these to a calibrated prediction set, and the rover asks for help whenever that set contains more than one option. Reproduced from Ren et al. [3]. 11
2.4. The introspective planning pipeline: a knowledge base of explained past examples is built offline, and at deployment the most similar example is retrieved to inform the LLM's confidence for each candidate step. Reproduced from Liang et al. [4]. 13

3.1. The compute split. The three models and the dialogue run on the brain. The Pi runs only what must keep working without them. 18
3.2. One command from speech to report. Shaded blocks call a model. The dashed block drives the rover. Dashed arrows show the scene digest feeding the two checks that need to know what is in the room. 20
3.3. The dialogue phases up to the start of a mission. A rejection returns to `CLARIFYING` with the person's reason added to the history. 24

5.1. (a) Plans with the correct visit order out of three runs, against chain depth, at three context settings (points are offset horizontally for legibility). (b) Projected tokens per planning call against depth, measured to depth 30, with the 1024 and 2048 limits marked. 34

List of Tables

1.1. Levels of command complexity considered in this project. 3

3.1. Functional (FR) and non-functional (NR) requirements, with the objective each one serves. 17
3.2. What the design provides for each command level. 18
3.3. The eight light states. 21

5.1. Physical rover outcomes per command level, final batch (Gemma 4 E4B for planning and vision). 30
5.2. Sonar reading after the rover stopped, per case (mm). Threshold 300 mm. . . 31
5.3. Vision-model distance estimates against measured distance (visible frames only). MAE is the mean absolute error; the rank correlation is not defined for Qwen in bucket mode because its output was constant. 32
5.4. Virtual chains by kind and depth. Each cell gives correct plans and completed missions out of the runs ("Corr., compl."), and the median planner latency in seconds ("Plan"). 33
5.5. Correct plans out of three runs, by context setting and chain depth (Gemma 4 E4B). 33

Nomenclature

Variables and Functions

Constants

c	A constant.

Functions

f	A function.

Variables

$\mathbf{x}$	A variable.

Acronyms and Abbreviations

API

CPU

FPS

HRI Human-Robot Interaction

IMU

LLM Large Language Model

SLAM

STT Speech-to-Text

VLM Vision-Language Model

Definitions

S

Student is an entity needing a thesis to transcend the state of being a student.

Chapter 1

Introduction

1.1 Background

A robot that takes spoken instructions removes two obstacles that make most mobile robots difficult to operate day to day. Typically a mobile robot operates using a screen to navigate, and an operator trained on its command set. This matters even more for a small indoor rover, because the tasks it can be asked to do, such as checking whether a door is open or looking at what is on a shelf are exactly the kind of tasks a person would rather say out loud than type into an application.

Large language models (LLMs) are neural networks which are based on the transformer architecture, and are trained on very large collections of text. Through this training the LLM's learns to generate the next word in the sequence of text given the previous words in the sequence. This makes LLM's well suited to interpreting everyday instructions as well as have a huge general knowledge base of the world. For example, when given a natural-language command and a description of what a robot can do, an LLM can break the command into a sequence of simple actions called a (RAP). This RAP can be used by any robot, and can be made up of actions that the robot is able to perform, without a developer having to hand-code a fixed list of recognised phrases [1]. Ambiguous commands can also be resolved through clarifying questions, when a command leaves out information, the model can ask the user follow-up questions before it commits to a plan. This exchange, in which the user and the robot take turns to communicate until the task is clear, is referred to in this report as a *multi-turn dialogue* [2].

Handing the planning to a language model introduces two problems. The first problem is that spoken commands are often ambiguous. So systems that always acts on its first interpretation will act on the wrong one often enough to matter [3]. The second problem is that language models hallucinate. They produce fluent, confident output that does not correspond to anything real, and this becomes more likely as a task is broken into more steps [4]. The techniques that detect ambiguity or hallucination before a plan is set

in stone can also be used while the plan is being carried out, by monitoring the robot's observations for signs that something has gone wrong [5, 6].

The systems that demonstrate these techniques typically rely on large models hosted in the cloud and accessed over the internet. These systems also typically run on robots with rich sensing such as , an inertial measurement unit (IMU) and wheel encoders that measure how far each wheel has turned. While both assumptions are reasonable in a research laboratory, they do not hold in every setting in which a voice-commanded robot would be useful.

A use case of running local only models on a robot is in a private home. Every command sent to a cloud-hosted model, and every camera frame used to check the robot's progress. These commands and images leaves the home and is processed on a third party's servers, which many households would not accept for a device that moves through their living space and records it. A home robot also has to be affordable, which rules out and a powerful on-board computer and leaves a camera as the most practical source of information about its surroundings. Low-cost drive platforms often also lack wheel encoders and an IMU, and their wheels slip enough that a command such as "drive forward one metre" cannot be trusted to be carried out accurately. In this setting, the language models must run on hardware the household already owns, such as a desktop computer with a consumer graphics card, and the robot has to rely on its camera to know whether its plan is working. Where privacy is not a concern and a reliable internet connection is available, a cloud-hosted model remains the better choice, since it is larger and more capable than any model that fits on consumer hardware.

1.2 Problem Statement and Research Questions

There is not a full understanding of whether language and vision models small enough to run on a single consumer desktop computer and can reliably turn a vague spoken command into a plan that a low-cost indoor rover can carry out using only its camera and simple on-board sensors. It is also not known how much complexity such a command can contain before the locally hosted models begin to produce plans that do not match the command, the room or the rover's capabilities. Existing work either assumes large, cloud-hosted models, or does not connect a spoken clarification dialogue to a physical robot that must wait for the user's confirmation before it moves.

In my skripsie, consumer hardware refers to a gaming desktop computer with an AMD Ryzen 5 3600 processor, 16 GB of system memory and an NVIDIA GeForce GTX TITAN X graphics card with 12 GB of video memory, representative of a mid-range gaming computer several years old. All locally hosted models, for speech recognition, planning and vision. This all must run within the memory of this machine.

To make these questions measurable, the commands the rover must handle are grouped

into the levels of increasing complexity listed in Table 1.1.

Table 1.1: Levels of command complexity considered in this project.

Level	Type	Example
L1	Single action	"Turn left."
L2	Sequence of actions	"Drive forward, turn left, then drive forward again."
L3	Perception-dependent	"Drive forward until you see the door."
L4	Conditional	"Check whether the door is open. If it is, go through it; otherwise, come back."
L5	Vague	"Go and see who is in the kitchen."

RQ1 Can locally hosted models, running on the consumer hardware defined above, turn a vague spoken command into a plan that the rover can execute, and which levels of command in Table 1.1 can they handle reliably?

RQ2 For each level of command that the models can handle, how many steps can be chained into a single command before the plan starts to include hallucinations, meaning actions, objects or locations that are not supported by the command, the room or the rover's capabilities?

In both questions, the rover's camera is its main source of feedback about the world. A vision-language model (VLM) uses camera images to check the plan against the room before the rover departs, and to confirm the rover's progress while the plan is carried out.

1.3 Objectives

The main objective of this project is to build and evaluate a fully local pipeline that turns a spoken, potentially ambiguous command into a plan an indoor rover can carry out, using models small enough to run on a single consumer-grade machine rather than a cloud API. To reach this, the project must:

1. Assemble a rover from off-the-shelf components on which the rest of the system can be tested, with a camera as its main sensor and an ultrasonic distance sensor and line sensor in supporting roles.

2. Determine which speech-to-text models can transcribe spoken commands accurately and quickly enough when run on the consumer hardware, including for speakers of South African English and Afrikaans.

3. Implement the rover's capabilities as a set of modular functions, and determine which levels of command in Table 1.1 a locally hosted planner can reliably map onto these functions.

4. Implement a multi-turn dialogue that resolves ambiguity in a command before a plan is generated, followed by a spoken confirmation step in which the rover reads its plan back to the user and only starts moving once the user agrees.

5. Ground each plan in the actual room, using a walkthrough scan recorded beforehand and camera frames checked during execution.

6. Measure and minimise the time taken from the end of a spoken command to a confirmed plan, without reducing the correctness of the plan.

7. Evaluate how the correctness and executability of plans change as commands move up the levels in Table 1.1 (RQ1).

8. Evaluate how the number of steps chained in a command relates to the rate of hallucination (RQ2).

9. Compare the locally hosted pipeline against equivalent cloud-hosted models in terms of plan correctness and response time.

1.4 Scope and Limitations

Since this project is concerned with turning vague spoken commands into robot action plans. Its scope is limited to the pipeline from speech to executed plan. This includes the transcription of speech, the clarification of ambiguous commands through spoken dialogue, plan generation and verification, and the spoken confirmation step. Navigation is used only as a means of carrying out and testing the plans, and improving navigation itself is not an aim of this project.

Before the rover is used in a room, the room is recorded once in a short walkthrough video. A VLM describes the key frames of this video in text, producing a written summary of the objects in the room and their layout. This summary is given to the planner, so that plans refer only to things that are actually present. Selected frames are also stored with their descriptions as a reference set, against which the rover's live camera frames are compared during a mission to check that it is where the plan expects it to be. The scan is not turned into a three-dimensional model or map, and building a general-purpose mapping system is outside the scope of this project.

Cloud-hosted models are used only as a benchmark for the locally hosted pipeline, and are not part of the system under test.

The rover is deliberately limited to a camera, an ultrasonic distance sensor and a line sensor, with no wheel encoders, IMU or . This reflects the low-cost home robot described in Section 1.1, and forces the pipeline to rely on the camera to determine whether a plan is succeeding, which is the capability this project sets out to test. As a result, movement is carried out without precise measurement of distance or heading, and commands that require precise distances beyond what the ultrasonic sensor can measure are not supported.

Similarly, the system is not designed to operate in arbitrary rooms or with arbitrary commands. Evaluation is limited to a small number of test rooms and a fixed set of test commands at each level in Table 1.1, which is sufficient to answer RQ1 and RQ2. The results therefore describe the pipeline's behaviour on the cases tested. Rather than guaranteeing its performance in other rooms or with other commands.

1.5 Report Outline

The remainder of this report is structured as follows: a review of related work in task planning, ambiguity resolution, and hallucination in robotic systems; the design of the speech-to-plan pipeline and its clarification loop; the rover platform and the interface between the planning software and the hardware it controls; the test methodology, including the simulated executor used in place of physical hardware; the results and discussion for each research question; and a concluding summary with recommendations for future work.

Chapter 2

Literature Study

2.1 Language Models as Robot Task Planners

Large language models carry a great deal of general knowledge about the world, but that knowledge is not grounded in what any particular robot can actually do. An LLM asked to "tidy the kitchen" has plenty to say about what tidying a kitchen involves, and none of it accounts for whether the robot in front of it has an arm, what that arm can reach, or whether "the sponge" refers to something the robot has ever seen. To Close the gap, connecting the model's reasoning to a specific robot's actual capabilities, is what the systems in this section have in common, even though they close it in different ways.

SayCan [1] grounds the model by constraining its output rather than trusting it outright. The LLM proposes a set of candidate next actions drawn from a fixed library of pretrained skills, but each candidate is re-scored by a separately learned affordance function. that estimates how likely that skill is to succeed given the robot's current state. The action that is both something the LLM would suggest and something the robot can plausibly do is the one that gets executed. Grounding, in this framing, is a second model checking the first, not something the LLM is trusted to know on its own.

Code as Policies [7] takes a different route. Rather than picking from a fixed skill library, the LLM writes code, calling perception functions, control primitives, and other functions it has been shown examples of, composed however the instruction requires. This lets it handle instructions no one anticipated in advance and assign precise parameters (a velocity, a threshold) to descriptions that were only ever given in words, at the cost of needing a language model capable of writing correct, composable code rather than just picking from a short list.

Inner Monologue [8] addresses a limitation both of the above still have: a plan committed to once, before anything happens, has no way to adapt as the situation unfolds. It closes the loop by feeding the robot's ongoing observations, success and failure signals, scene descriptions, even a human's spoken interjection, back into the language model's own

reasoning process, so it can revise what it does next in light of what actually happened rather than what it expected to happen.

None of these three were built for tasks longer than about ten actions. Hori et al. [2] target that gap directly, arguing that long-horizon tasks fail for a different reason than short ones: the command usually leaves out information the model would need to plan the whole thing correctly, and no amount of re-scoring or code generation fixes a plan built on a wrong assumption. Their method resolves this through what they call active modification, the model asks clarifying questions before committing to a plan, combined with passive modification, refining the plan afterwards against visual feedback. This is the most direct ancestor of the clarification loop this project builds: extending plan length by first closing the information gap in the command, rather than by making the underlying model larger or its skill library richer.

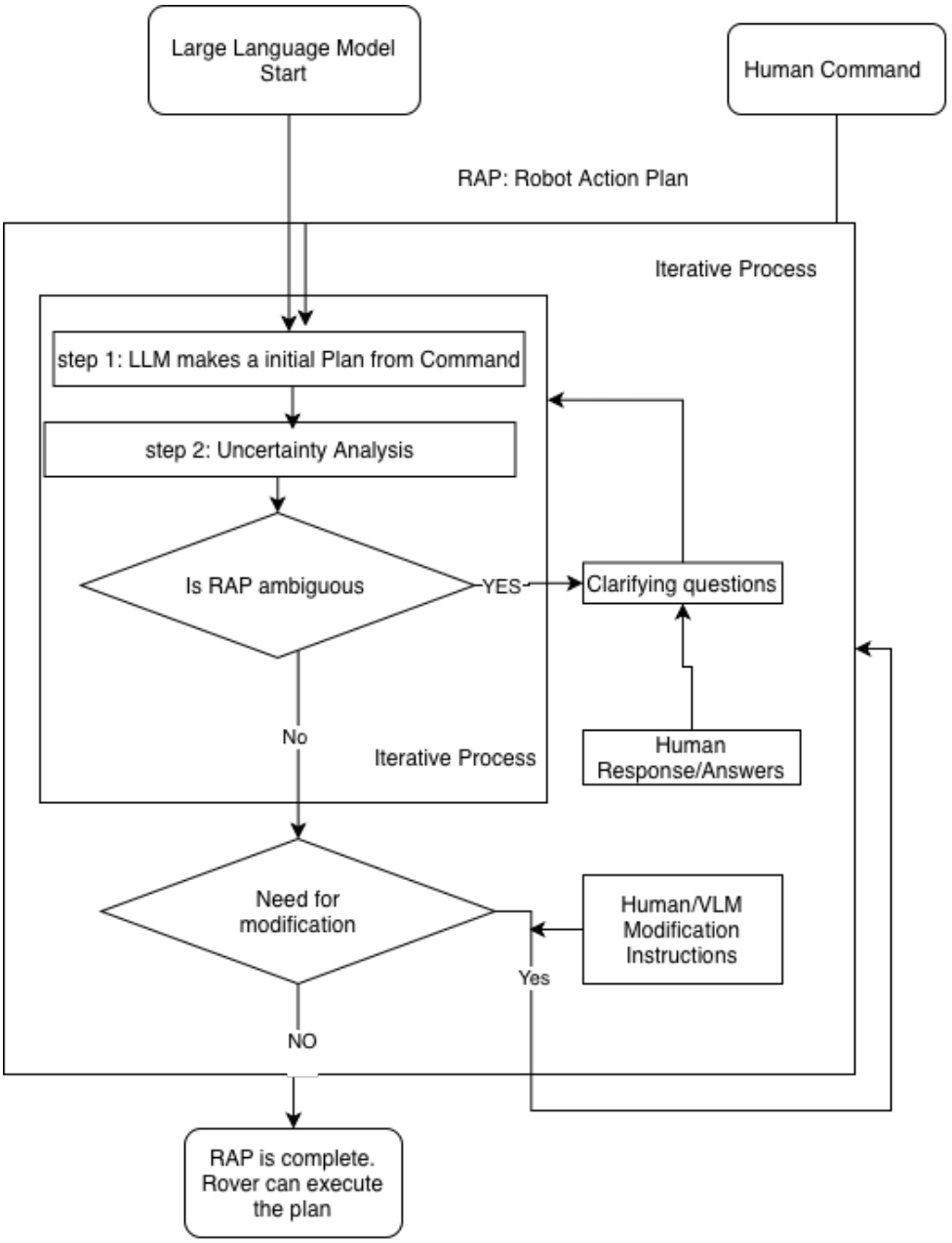


Figure 2.1: Process overview of RAP generation before rover departures

Last point Code as Policies' approach assumes a language model capable of writing correct, composable code against a rich API, which is a reasonable assumption for a cloud-hosted model that has more computing power but not for the small, locally hosted planner this project depends on for RQ1.

2.2 Grounding Plans in a Bounded Action Set

SayCan and Code as Policies set the standard way to connect an LLM's plan to what a robot can actually do. SayCan [1] pairs the LLM with a separately learned value function for each robot skill. The language model scores how relevant an action sounds for

the instruction. The value function scores whether the robot can do it right now. The two scores are combined, so the action picked is both sensible and physically possible. Figure 2.2 shows this with a concrete instruction, "how would you put an apple on the table?" The LLM alone ranks *find an apple*, *find a coke*, and *find a sponge* almost the same, since all three are plausible first steps for some task. The value function breaks that tie: *find an apple* and the other two score identically on the language side, but only the apple is actually visible in the robot's current scene, so the combined score, log-probability from the LLM multiplied against the affordance probability, pushes it to the top of the list. Nothing about the instruction told the model an apple was present. The value function is what supplied that.

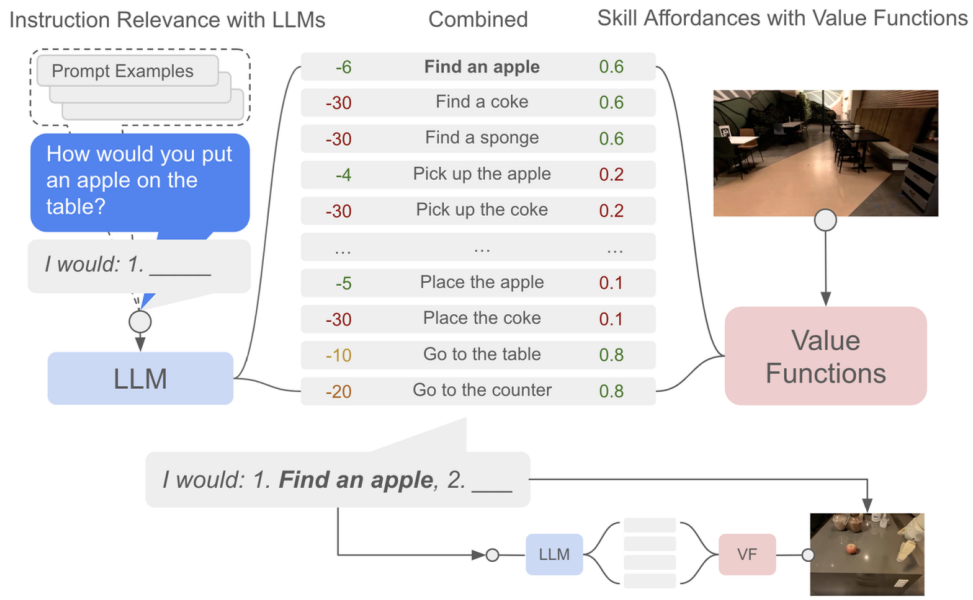


Figure 2.2: SayCan combining language-model relevance scores with value-function affordance scores to select the next skill, reproduced from Ahn et al. [1].

Ahn et al. formalise this selection rule as

$$\arg\max_{\pi \in \Pi} \; p(c_\pi \mid s, \ell_\pi)\, p(\ell_\pi \mid i) \tag{2.1}$$

where: Π is the set of available skill

ℓ_π is skill π's textual label,

i is the instruction,

s is the current state,

c_π is the event that π completes successfully.

$p(\ell_\pi \mid i)$ is task-grounding, the language model's estimate of how well a skill matches the instruction, and $p(c_\pi \mid s, \ell_\pi)$ is world-grounding, the value function's estimate of whether that skill would succeed from the current state.

These are exactly the two columns in Figure 2.2, the LLM's log-probabilities are $\log p(\ell_\pi \mid i)$, and the 0.6/0.2/0.1/0.8 values are $p(c_\pi \mid s, \ell_\pi)$.

Code as Policies [7] solves the same problem a different way: the LLM writes Python that calls a fixed set of perception and control functions directly. If the code does not compile against that API, the step cannot run.

Both approaches give the planner a fixed set of actions it cannot go outside of. This project's capability vocabulary in `planner.py` plays the same role: a step whose target or action falls outside that vocabulary is rejected rather than passed through to the rover.

Neither approach checks what happens after a step runs. Inner Monologue [8] adds that check on top of a SayCan-style setup: after each action, the robot gets an observation back, a success or failure signal, a scene description, or something the human said, and the next step is planned using that observation instead of the original plan alone. Plan a step, act, observe, replan. This project's mid-mission execution follows the same loop: each frame sent over `/ws/execution` is what the plan gets checked against after each step, not only before the mission starts, allowing the plan to respond to drift the rover has actually accumulated rather than committing to a path decided before departure.

2.3 Clarification and ambiguity in human-robot interaction

A spoken instruction only resolves to a single rover action if the rover's observations rule out every other reading of it. "Bring the bowl" is unambiguous with one bowl on the counter and ambiguous with two. Hori et al. [2] describe two ways to correct a task plan once this kind of ambiguity shows up: passive modification, where a person reviews the plan and points out what is wrong, and active modification, where the planner judges its own uncertainty and asks a question before acting.

Passive modification is accurate, since a human checks every step, but it is repetitive. The plan has to be re-issued and re-checked each time something is off. Active modification moves that judgment onto the model, so the human is only asked to step in when the model is actually unsure.

Ren et al. [3] give active modification a statistical guarantee, in a method called KnowNo, and Hori et al. cite it directly as a working example of the approach. KnowNo does not let the planner commit to its single highest-scoring next step. The planner's raw confidence scores for each candidate step are passed through a conformal prediction layer, which converts them into a calibrated set of steps that are all plausible enough to keep. A singleton set means the rover proceeds on its own. A set with more than one step means the instruction was ambiguous given what the rover can observe, and the rover asks a clarifying question instead of guessing. Figure 2.3 shows the example used in the paper.

It is told to place the bowl in the microwave with a metal and a plastic bowl both on the counter, the two matching steps score close enough that neither clears the calibrated threshold on its own, so the system asks which one, plastic or metal, instead of picking one.

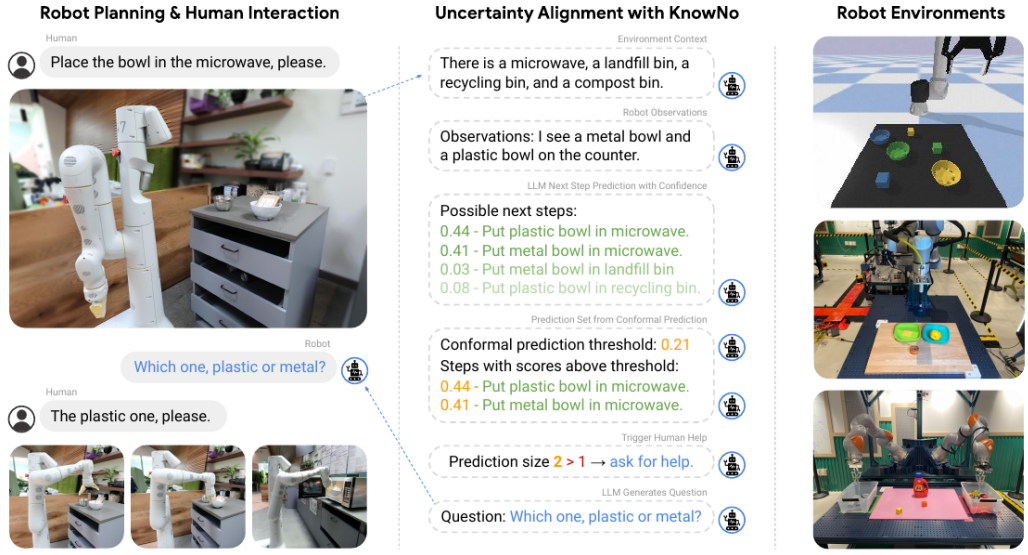


Figure 2.3: The KnowNo pipeline: an LLM planner proposes next steps with confidence scores, conformal prediction narrows these to a calibrated prediction set, and the rover asks for help whenever that set contains more than one option. Reproduced from Ren et al. [3].

The calibration works as follows. Given a held-out calibration set of N examples, each with a true label y_i and a model score $\hat{f}(\tilde{x}_i)_{y_i}$ for that label, a nonconformity score is computed for every example:

$$\kappa_i = 1 - \hat{f}(\tilde{x}_i)_{y_i}. \tag{2.2}$$

The calibrated threshold $\hat{q}$ is then set to the

$$\frac{\lceil (N+1)(1-\epsilon) \rceil}{N} \text{ empirical quantile of } \kappa_1, \ldots, \kappa_N, \tag{2.3}$$

where ϵ is a user-chosen error tolerance. At test time, the prediction set kept for a new instruction is every label whose score clears this threshold:

$$C(\tilde{x}_{\text{test}}) = \left\{ y \in Y \;\middle|\; \hat{f}(\tilde{x}_{\text{test}})_y \geq 1 - \hat{q} \right\}. \tag{2.4}$$

Conformal prediction guarantees that this set contains the correct label with the user-specified probability,

$$P\Big(y_{\text{test}} \in C(\tilde{x}_{\text{test}})\Big) \geq 1 - \epsilon. \tag{2.5}$$

This guarantee is what makes the threshold usable in practice: instead of picking a confidence cut-off by hand and hoping it behaves sensibly, the rover's decision to act alone comes with a stated bound on how often that decision is correct.

2.4 Locally hosted and quantised models

Sections 2.1 to 2.3 assume the rover can call out to a large, cloud-hosted LLM. A rover cannot always guarantee that access: network connectivity is not reliable in every test environment, and the round-trip latency to a cloud endpoint works against the response time a voice-commanded system needs to feel usable. Running the planner locally, on hardware the rover carries itself, removes both problems, but restricts the model to whatever fits in on-board memory.

Quantisation is the standard technique for making a model fit that constraint: model weights, and in some cases activations, are stored using fewer bits than the 16 or 32 bits used at training time, at some cost to numerical precision. Liu et al. [9] define uniform quantisation of a tensor $\mathbf{X}$ to b bits as

$$\hat{\mathbf{X}} = \mathcal{Q}(\mathbf{X}; b) = s \cdot \Pi_{\Omega(b)}\left(\frac{\mathbf{X}}{s}\right), \qquad s = \frac{\max(\mathbf{X}) - \min(\mathbf{X})}{2^b - 1}, \tag{2.6}$$

where

* $\Omega(b) = \{0, 1, \ldots, 2^b - 1\}$ is the set of representable integer levels
* $\Pi_{\Omega(b)}(\cdot)$ rounds to the nearest one.

This is applied at different bit-widths for weights and activations, written for example as W8A8 (8-bit weights, 8-bit activations) or W4A16 (4-bit weights, 16-bit activations); A lower bit-width gives a smaller model and a coarser step size s, and therefore more rounding error per value.

Husom et al. [10] benchmark quantised LLMs directly on edge hardware and report that quantisation lowers energy use and inference latency substantially, at a small cost to output accuracy on straightforward tasks. Liu et al. [9] show that this cost is not constant across task types. Accuracy degrades faster as bit-width drops on tasks that require chaining several reasoning steps together than on single-step tasks. The planning step described in Section 2.1 is exactly that kind of multi-step reasoning task, so the bit-width chosen for a locally hosted planner in this project is not only an efficiency setting. It is a direct trade-off against how reliably the planner reasons through a multi-step voice command.

2.5 Compositional depth and hallucination

The methods discussed in Sections 2.1 to 2.3 treat each planning step largely on its own. As a task plan grows longer and more compositional, two further problems appear that a single-step clarification mechanism does not cover: the LLM's reasoning can degrade over a long chain of steps, producing a confidently stated plan that is simply wrong, and the rover can encounter a scene it was never prepared for, one that no single faulty component causes but that the plan as a whole fails to account for.

Liang et al. [4] address the first problem by extending the conformal prediction procedure from Section 2.3 with a retrieval step. Rather than scoring each candidate plan from the LLM's raw confidence alone, their method, introspective planning, first builds a knowledge base of worked examples. For a given task and observation, an LLM is prompted to explain in writing why certain candidate actions are valid and others are not. At deployment, the system retrieves the most similar past example by embedding similarity and includes it in the prompt before the LLM scores its candidates, as shown in Figure 2.4. Grounding each new decision in a similar, explained precedent sharpens the confidence scores that conformal prediction operates on, which produces the same statistically guaranteed prediction set from Equations 2.4–2.5 while asking the user fewer unnecessary questions.

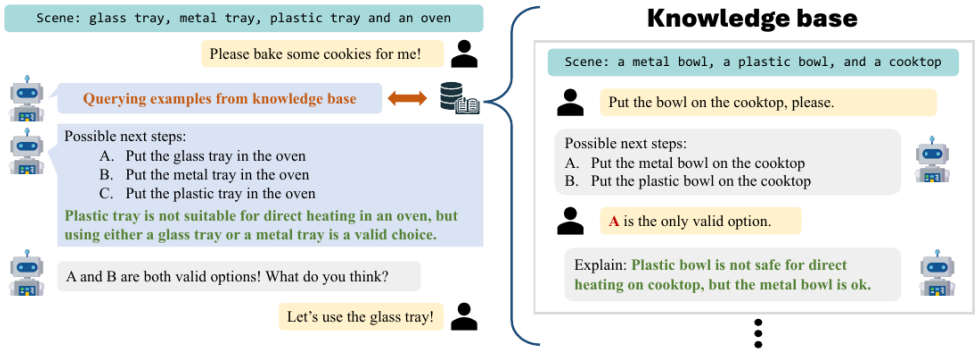


Figure 2.4: The introspective planning pipeline: a knowledge base of explained past examples is built offline, and at deployment the most similar example is retrieved to inform the LLM's confidence for each candidate step. Reproduced from Liang et al. [4].

The second problem, an unanticipated scene rather than an unreliable plan, is addressed by Sinha et al. [5] and Elhafsi et al. [6] from the direction of runtime monitoring rather than planning. Sinha et al. propose a two-stage monitor: a fast binary classifier screens every observation in an LLM embedding space, and only when it flags a possible anomaly does a slower, more capable LLM reason explicitly about what has gone wrong and which fallback plan to switch to. Splitting the monitor this way keeps the fast path cheap enough to run continuously, while reserving the expensive reasoning step for the moments it is

actually needed, which matters directly for a rover with the limited on-board compute discussed in Section 2.4. Elhafsi et al. apply the same idea of an LLM as a monitor to what they term semantic anomalies. A system-level failures, like an autonomous vehicle braking for a stop sign printed on a billboard. That does not come from any single broken component, but from a gap in the system's contextual reasoning.

Together, these three methods extend the single-step clarification of Section 2.3 along a different axis: introspective planning improves the confidence estimate feeding a single decision, while AESOP and semantic anomaly detection catch failures a confidence score never gets the chance to be wrong about, because the plan itself is being carried out against a scene the planner misjudged.

2.6 Speech recognition for South African English and Afrikaans

Every method discussed so far assumes the instruction d_i handed to the LLM planner is already correct text. For a voice-commanded rover, that text comes from a speech-to-text stage first, and an error there becomes an error in the planner's input before any clarification mechanism gets a chance to act on it. South African English and Afrikaans are both under-represented in the training data of general-purpose speech recognisers, which makes this an upstream risk specific to a South African test environment.

Kamper et al. [11] quantify how much accent variation costs a algorthmn used for recognising speech, trained without accounting for accent variation. Working with 31.78 hours of speech across five South African English accents (Afrikaans English, Black South African English, Cape Flats English, White South African English, and Indian South African English), they compare three acoustic modelling strategies: a separate model per accent, a single model pooling all accents together, and a multi-accent model that shares acoustic structure across accents where it helps while keeping accent-specific distinctions where it does not. The multi-accent model reached 82.78% word accuracy, against 81.53% for accent-specific models and 81.52% for a single pooled model, a difference the authors report as statistically significant. Black South African English stood out as the one accent that still benefited from being modelled separately rather than pooled, which the authors attribute to it being acoustically the most distinct of the five.

Jacobs et al. [12] work at the opposite end of the resource spectrum: fine-tuning Whisper on Afrikaans and isiXhosa oral narratives spoken by four- and five-year-old children, using only five minutes of transcribed in-domain child speech. They find that adding in-domain adult speech, especially combined with voice conversion to make it sound more child-like, gives the largest accuracy gain of any strategy tried, and that parameter-efficient fine-tuning helps Afrikaans but not isiXhosa, since isiXhosa is more

severely under-represented in Whisper's original training data.

Together, these results point at the same conclusion from different directions. Local accents and languages measurably degrade off-the-shelf speech recognition, and the fix in both cases is not more general-purpose training data but a small, targeted amount of in-domain or in-accent data. For a rover meant to take spoken commands from South African English or Afrikaans speakers, this is a design constraint on the speech-to-text stage that sits upstream of every planning method discussed in Sections 2.1 to 2.5, not something those methods can correct for after the fact.

2.7 Research gap

Sections 2.1 to 2.6 cover five separate strands of work: grounding an LLM's plan in what a rover can actually do (Sections 2.1-2.2), asking for clarification when an instruction is genuinely ambiguous (Section 2.3), catching the failures that a confidence score alone will not (Section 2.5), running the planner on hardware small enough for a rover to carry (Section 2.4), and recognising speech accurately in a South African accent or language (Section 2.6). Each strand has been developed and evaluated largely on its own.

The methods in Sections 2.3 and 2.5 that provide statistical guarantees on plan correctness, KnowNo and introspective planning, are demonstrated with large, cloud-hosted LLMs. Section 2.4 shows that quantisation, needed to run a planner locally, costs accuracy disproportionately on exactly the kind of multi-step reasoning these planners perform, but neither Ren et al. [3] nor Liang et al. [4] evaluate their calibration procedure on a quantised, locally hosted model. Whether a conformal prediction threshold calibrated on a full-precision model still gives a valid guarantee once that model is quantised down to fit on-board memory is not addressed by either strand of work.

Similarly, none of the planning or clarification methods in Sections 2.1 to 2.5 are evaluated with speech input, let alone speech in a South African accent or in Afrikaans. Section 2.6 shows that accent and language mismatch measurably degrades transcription accuracy, which means an instruction d_i reaching the planner in this project's setting carries a source of error the literature above does not model at all. An ambiguity introduced by a transcription error is not the same problem as an ambiguity in the underlying instruction, and it is unclear from the existing literature whether a clarification mechanism designed for the latter behaves sensibly when faced with the former.

This project sits at the intersection of these gaps: a voice-commanded indoor rover that must plan and clarify using a locally hosted, quantised LLM, taking speech input from South African English or Afrikaans speakers. No single work reviewed in this chapter evaluates this combination, and the remainder of this report investigates how the planning, clarification, and speech recognition components described above hold up once combined on hardware a rover runs on.

Chapter 3

System Design

3.1 User and Functional Requirements

The end user in mind is someone who is not staring at a screen. It is someone who would be able tel a indoor rover what to do using speech. They are also not necessarily going to watch a screen to see if the rover is on track. They can say things like 'Go look at the box on the room' or 'what is on the floor under the bed'. The rover will have to clarify if there is multiple boxes or where the bed is in the room. Ask any questions to resolve ambiguity and then go and execute it's plan.

In the table below is the requirements in Table 3.1. Each one traces back to an objective in Sections 1.3.

Since there is no odometry sensors on the provided rover. Every distance covered by the rover is a open-loop estimate. The camera and sonar sensors are used to correct this estimate. The rover travels and moves in open-loop steps but gets corrected once a step is completed or a obstacle is encountered by camera and ultrasonic sensors.

To Test how well the plans are generated. I made a table with command levels in Table 1.1 each need something different fromt he design, and Table 3.2 says what. These mappings were made as the project progressed and the design evolved. For example, at first I did not think it was necessary but it evolved as the project got more scope.

3.2 System Architecture

3.2.1 Off-board Inference

The rover used is a HiWonder TurboPi with a Raspberry Pi 5 and about 4GB of RAM. The Turbo Pi has and expansion module called the HiWonder Echo pro microphone and audio adapter connected via USB The planner, whisper translator module and Vision

Table 3.1: Functional (FR) and non-functional (NR) requirements, with the objective each one serves.

ID	Requirement	Objective
FR1	Take the command by voice and transcribe it locally.	2
FR2	Before planning, check the command against the room and ask the person a spoken question when the command is ambiguous, up to a fixed number of questions.	4
FR3	Produce a plan that uses only actions the rover has and objects the room contains.	3, 5
FR4	Read the plan back and move only after a spoken confirmation. A rejection must lead back to the dialogue, not end it.	4
FR5	Carry out the plan one step at a time, if a obstacle is in the rovers way it should stop without consulting a model.	1, 3
FR6	Use the camera to check progress and arrival during a step.	5
FR7	Report the outcome of the rover if it completed the steps and if it was unsure about anything.	4, 7
FR8	When a step fails, ask the person what to do next and carry on from where the rover is. This is for failure only, never for ambiguity.	4
FR9	Log every model call and decision made to help with debugging.	6, 7, 8
FR10	When a command contains a condition, look once to decide it, run only the matching steps, and ask the person when the camera cannot tell. Never guess a branch.	3, 7
FR11	When the person asks what is near a target, report what is around it and not only whether the target was reached.	3, 5
NR1	Run planning, vision and speech on consumer grade hardware, e.g. a Ryzen 5 3600 with 16 GB RAM and a 12 GB GPU. Cloud models are allowed only as a benchmark.	9
NR2	Keep the rover stoppable when the network or a model is slow or gone.	1
NR3	Use only a camera, an ultrasonic sensor and a line sensor. No encoders, IMU or lidar.	1
NR4	Let the planner or vision model be swapped by configuration, with no change to the pipeline code.	9
NR5	Show the person what state the rover is in without a screen, using onboard lights	4

Table 3.2: What the design provides for each command level.

Level	Command type	Mechanism
L1	Single action	One `move` or `turn` step, read back and confirmed.
L2	Sequence of actions	Ordered steps, each finished before the next starts.
L3	Perception-dependent	`approach` to find and reach a named object (Section **??**), or `follow_line` to a visible end point.
L4	Conditional	A `check` step with tagged branches (Section **??**).
L5	Vague	The clarification loop before departure (Section 3.4) and a survey of the surroundings when the command asks for one (Section **??**).

model runs on a separate host machine. Keeping only the motion and sensing on the Pi itself. The reason for this configuration is due to size. The 3 models together take about 10GB of RAM and if that is excluding the stack required to drive the PI and control the camera. There is physically enough room to have a reasoning model on this hardware. Not yet atleast.

Running the models on a seperate machine with more hardware capability also means that I can swap a model by changing one environment variable, which the cloud comparison depends on.

I call the seperate machine hosting the models 'the brain'. It communicates to the turboPi over a rosbridge on a port that it exposes. Using a library called 'roslibpy' (Figure 3.1). This allows for commands from the brain like: The plan, the keyframes, vision results, and questions for the user to be sent to the rover. The Camera frames, sonar readings And step results come back to the brain.

since the brain hosts all the model decisions and state of the current plan. If there is a network drop the rover can still continue operating with the last known plan. It just cannot ask for clarification or get new plans.

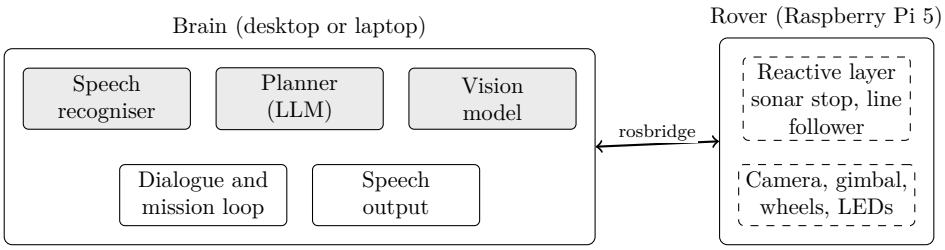


Figure 3.1: The compute split. The three models and the dialogue run on the brain. The Pi runs only what must keep working without them.

The code boundary is one method, `RoverController.execute_step`. The brain hands it one step and gets back `ok`, `blocked` or `halted`. Everything above that method is the same for the real rover and for a simulated one, so I can run the same dialogue and mission code in simulated room which is a constructed room from a JSON file before putting it on hardware.

The default configuration is fully local. Speech recognition is `faster-whisper` with the `fluister-turbo` model, the planner is `gemma4:e4b` `gemma4:e4b`, and the vision model is `qwen2.5vl:3b` or `gemma4:e4b`. Both are served by Ollama with a context of 8192 tokens.

Speech output uses Piper, a small neural text-to-speech engine that runs locally. It turns text into a waveform with a voice model stored as an ONNX file, and I use the `en_US-lessac-medium` voice. I chose it for three reasons. The rest of the pipeline is local, so the rover's voice has to be local too (NR1), and a cloud speech service would put a network call and a third party into every spoken reply. Piper also runs on the CPU so it does not compete with the Vision and planner models for VRAM. One thing to note if I use 1 model for both vision and reasoning, I would have more VRAM at my disposal.

The 8192-token window is a constraint I check instead of assuming. Every clarification turn and every recovery turn adds text to a prompt. The small local model usually silently truncate plans without raising any error. A separate offline tool, `context_budget.py`, counts the tokens each part of the prompt takes (fixed instructions, room digest, one clarification turn, one recovery turn) so that I know how many turns fit before the window is the limiting factor.

3.2.2 Two-rate Control

Everystep or run requires the rover to do a vision-model call. In order to process, where it is at the current time and if it is still on track. A vision-model call takes about 4.78 s on my hardware. If the rover is using the line follower to control the driving, The steering of the rover cannot occur in a closed loop.

Instead there is a reactive layer on the turboPi itself, that came stock with it. It reads the sonar at 10-30 Hz and stop the rover if something is in the way. It also follows a line from the IR array uder the rover without consulting the models. The actual decision making engine runs on the brain at a pace of the models and hardware. It chooses the next steps, deciddes whether the rover has arrived at the desired destination. Where the closed loop system occurs is at every step boundary and at distinctive times during the rovers runs.

The reason it is design like this is because the rover never relies on the vision model to make a immediate decision on when to stop If a obstacle arises infront of the rover.

3.2.3 Pipeline

The Figure 3.2 gives you a the overview of the process of one command, given to the rover. A short walkthrough video is recorded of the room the rover will be traversing ing. The then breaks up the video of the room into keyframes (the number depends on the environment variable set). The vision model them lists all the objects it sees in the in each frame as well as any obstructions in the image. This is called the scene digest which can be called on by the model to understand the environment. It is capped at 8 major objects with each having 2 attributes for each object. The reason it was capped at 8 objects so that it fits the planners prompt without the dialogue.

I chose text over a map delibreatly. A map is a seperate system to create and keep updating and generating one is outside the scope of my Skripsie.

A spoken command by someone is then transcribed and enters the clarification loop (Section 3.4). If the command is not flagged as ambiguous the planner can turn is into steps using the actions and vocabulary/actions the rover can execute. (Section 3.3). The vision model then checks the plan against the digest for steps that are not able to be executed. If it comes back successful the plan is read back to the command giver and the rover only moves after they confirm the plan. The rover then run the steps one at a time and at the end reports are logged and spoken.

A Websocket will be used to carry the dialog sessions. `/ws/dialogue` will serve everything up until the confirmation, and `/ws/execution` servers all the communication during the mission run. I keep them apart so that the dialogue state and the mission state never share a single session.

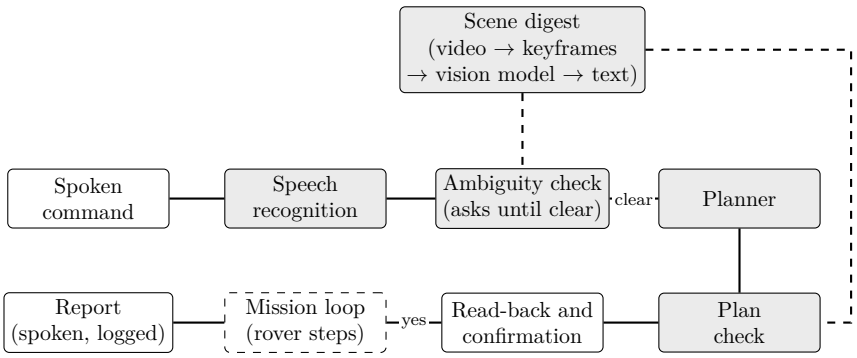


Figure 3.2: One command from speech to report. Shaded blocks call a model. The dashed block drives the rover. Dashed arrows show the scene digest feeding the two checks that need to know what is in the room.

3.2.4 Status Lights

To provide the user with some feedback on the rover state throughout the mission so that they can tell what the rover is doing. The Pi has RGB LEDs the expansion board as well

as the ultra sonic sensor ring. I used them as a state display (Table 3.3). During each phase of the mission execution as well as the dialogue session before a mission is executed. Each light color is tied to one state. If the light setting failed it should not hinder the rovers execution therefore Rovers that have no LEDs inherit a no-op, and one setting (`RGB_ENABLED`) turns the whole thing off.

Table 3.3: The eight light states.

State	Pattern	Meaning
listening	steady blue	Waiting for a command or an answer.
thinking	red and green alternating, 0.5 s	A model is working.
confirm	steady amber	A plan has been read back and the rover is waiting for yes or no.
executing	steady green	Carrying out a step.
searching	steady cyan	Looking for a target.
awaiting_guidance	magenta, 1 s on and off	Stopped and waiting for the person to say what to do next.
arrived	three green flashes, then steady green	A target has been reached.
blocked	steady red	The mission has stopped and cannot continue.

The mission logs also log the changes with timestamps, therefore allowing one to reconstruct the mission run later on or use it for debugging.

3.3 The Capability Vocabulary

Since these planners are LLMs and have generalized knowledge, a planner based on a LLM may write any step it likes and will also write steps that the rover cannot execute. Commands like 'bring me that bottle' invites a `pick_up` step. Since our rover has not arm. I gave the planner a fixed list of actions and in the prompt describe which exists and what each of these actions means.

The list depends on the rover, so it lives in a planner profile. If a different hiWonder Rover is chosen, the actions can be overridden using a different configurtion of Planner. The Real rover has the following actions: `follow_line`, `move`, `turn`, `approach`, `scan`, `observe`, `stop` and `report`. Every step that the rover executes is a small JSON object with an id, an action and a target.

The simulated rover has the same actions except for `follow_line`. The `move` and `turn` step both carry a distance and an angle. The reason for that is the simple fact that the simulated rover can measure distance travel since the virtual room in which it travels has a define width and length. The real rover does not have this information about the room in which it is traveling. This was chosen in (NR3). The simulated rover also cannot

pick up things as it has no arm and cannot carry anything. These are all decided in the planner profile for the simulated rover.

There is a ninth action that is `check`, this allows the rover to 'look at the destination object and decide whether the condition is met' (Section **??**). This action only takes place when the command contains a words like: *if*,*whether*, *otherwise* or *unless*. The prompt then gets a extra block that explains the `check` step as well as the optional `when` option for the alternative. This gives the rover 2 options to choose from before executing the plan. The checked field is left out of everyother prompt for two reasons. The first reason is the extra line is more tokens take up in the already small context window of 8192-tokens. The second is if there is extra context that the rover does not need it can results in a plan being miss interpreted or inventing a condition that nobody spoke of. There is a limiting factor of deciding to use the `check` action with a word match is that conditional phrased in a different way e.g 'when you see the door stop', will not be translated as conditional by the planner. Conditional logic also only exists on the real rover's profile and not the simulated rover.

The first plan generated by the planner that has actions outside of the list is not rejected. There are three methods used to catch it afterwards. A step with an unknown action returns `blocked` status if the rover tries to run it. The reporting phase in the brain counts the out-of-vocabulary actions in each plan the rover executes in orer to measure how often the planner ignors the list (section 5.3). Before the rover goes off on its mission the person hears the entire plan. I chose to measure the first-plan failure instead of just reprompting it and fixing the plan. The only plan action that is edited on the fly is the `move` action when the rover is in free mode. It changes the action to `approach`, since these two actions are similar and the planner will try to use move instead of approach.

The plan generated by the planner when the rover halts and ask for further instructions. This plan is used in the mission as is and not read-back to the user. This means that every step is validated to make sure the action is in the profile of the rover. Any plan that fails this check is discarded (Section **??**). In free-roam mode the list for a replan leaves out the `follow_line`, since there is no floor line for it to follow.

3.4 Clarification Loop Design

3.4.0.1 Placement

Some spoken commands are often underspecified, and if the rover executes on the wrong assumption it could place it in the wrong place. The ambiguity is handeled at 3 distinct places.

Before the departure, the system asks for a clarifying question if the command is unclear or ambiguous. During the mission it does not stop to ask about any ambiguity. It

compares what is currently can see with the digest and corrects itself as best as it can (Section 3.4.3). When it reaches its destination it communicates what is still was unsure about. Any mid-mission ambiguity questions are left out on purpose since the command speaker might not be there to answer the question but there is an exception for if the rover halted and cannot continue.

A rover that tries its best to execute a plan and get to a distinace is far better than one which stops and waits for a reprompt on a ambiguity it spotted in it's current plan. This is different to what was seen in KnowNo [3] and Introspective Planning [4], which ask for help when their confidence drops.

3.4.1 State Machine

The command spoke moves through multiple phases in Figure 3.3. During the CLARIFYING, the vision model is used to determine whether the command is ambiguous. Using the given command and the scene digest. The vision models answer is in JSON with four fields: wether the command is ambiguous, why ,a question to ask, and a rewritten command. If it is ambiguous, the quesiton is spoken and the answer is added to the history. If it is clear, the rewritten command is sued from then on. The command only names objects and avoids words like 'left' or 'behind you'. Since those are relative to the camera view at the time of asking and can be completely wrong once the rover moves.

This clarifying loop is capped at five questions. If the command is still unclear after that, the system marks the session as capped and plans from the rewritten command if the model produced one after a question was answered or from the original command if no clarifying question was answered. The loop also drops a quetion it has already asked. If that is the only question that the model produces then there is nothing new to ask and the loop breaks and treats this as a cap moving onto the planning. A warning is added to the output plan when it is read-back to the user which starts with 'I am no fully sure I understoon', so the person knows the plan is not 100% certain. There is a cap to the number of questions asked since the user can get tired of answering questions or it can result in a endless ambiguous loop of questions and answers.

If the vision model ever fails to load. It is retired once and if it fails again, the person is told and asked to repeat their command again. Not having this fall back coould result in letting every command through.

3.4.2 Plan Check

After the planner model produces a plan. The vision model can then check it against the digest and returns whether it is workable and a list of concerns. If it says no, the planner runs once more with the concerns added as advice while the original goal stays as it was. The replan happens only once. Replanning a second time rarely fixes what the first replan

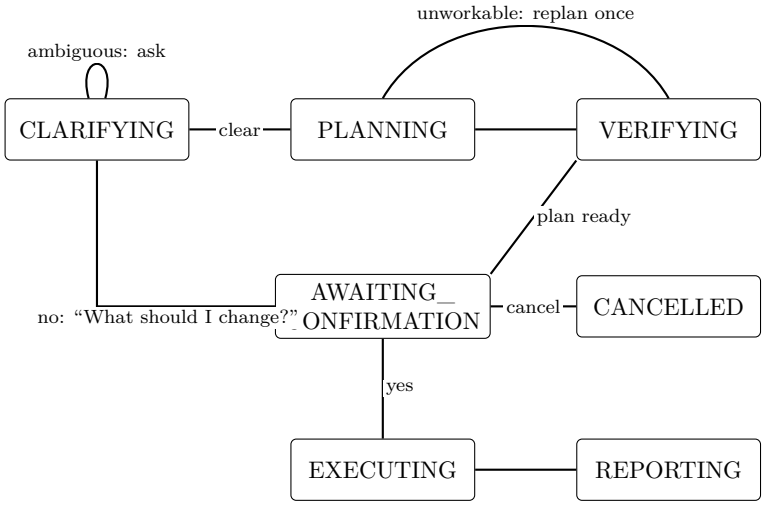


Figure 3.3: The dialogue phases up to the start of a mission. A rejection returns to `CLARIFYING` with the person's reason added to the history.

missed. If the plan checker itself is unavailable, the answer is neither yes nor no, and skipping the plan check is safer than the model hallucinating a concern.

3.4.2.1 Confirmation

The plan is read back as a short sentence per step. The rover then awaits for confirmation from the user and does not execute any steps until the confirmation is given. The answer provided is classified in two stages. All the keywords are passed over English phase list for all the possible combinations of 'no' and 'yes'. First the reject list is checked before the confirm list,'so 'no, dont't go' is not read as 'yes' because it contains 'go'. If nothing is classified as either 'yes' or 'no'. The rover counts that as unclear and the rover asks for the plan confirmation again.

If the plan is rejected the system asks what it should change and returns to `CLARIFYING` with the answer in the history, so that only one step coul be amended without starting over.

All commands can only be executed vocally however for debug purposes there is a typed input available. It can only be turned on if the environment variables is set for `ALLOW_TEXT_COMMANDS`.

3.4.3 Later Moments

As the rover is moving, the second moment is self-correction. As a step is executed, new frames are compared with the digest. A small module, `revision`, classifies what it sees into the following categories: no change, reroute, a material change or a block. This revision is done without a vision model but normal python code. The planner can be consulted to make a change, but the classifier decided whether the person has to be asked.

When the next step is exectued a reroute is applied. A material change moves the mission into `AWAITING_REVISION_CONFIRMATION`, and the rover waits for a spoken yes before it follows the new plan. The third moment is the report (Section **??**).

3.5 Navigation as Instrumentation

3.5.1 The Line Follower

Section 3.2.2 showed that the Mac-hosted models cannot sit in a steering loop. A VLM call takes about 15 s, and keeping a mecanum chassis on a line needs control at 10 to 30 Hz. I therefore left steering to the TurboPi's stock `line_follow` ROS2 node, which closes the loop on the Pi using the chassis-mounted 4-channel IR array. I used the IR array instead of the gimbal camera so the camera stays free for the VLM to look wherever the mission needs, regardless of where the chassis is steering (`rover_pi.py`, `aim()`).

`PiRoverController` does not reimplement steering. It only supervises the node on three things the node has no logic for itself:

- Arrival: the node's own `/line_follow/crossroads_stop` topic decides when the end of the line is reached, not a model's judgement.
- Obstacles: the ultrasonic sensor is polled independently of the line sensor. Two consecutive readings below the stop threshold pause the follower until the path clears, and then it resumes.
- Giving up: a wall-clock timeout. The stock node has no lost-line exit, so without one the rover would drive straight on indefinitely.

I also use a modified variant, `line_follow_corner.py`. When it takes a 90° bend instead of parking at a crossbar, it publishes a `/line_follow/corner` event. This lets a route be given as an ordered string of turns (e.g. `"L"`, `"R"`) and not only the straight-track default. It adds one constraint: a planned pause, such as a mid-drive look, must not land while the follower is mid-pivot. A `STOP` sent during the node's pre-turn nudge cancels the turn instead of the forward motion. `corner_guard_active()` reports a short window after each bend, and callers must defer their pause until it ends.

Halting the follower is where a naive design fails without any error. While running, the node republishes `/cmd_vel` at 100 Hz, so a single zero-velocity `Twist` is overwritten within 10 ms and the rover keeps driving. `halt()` therefore sends an explicit `STOP` service call to the node *before* the zero `Twist`. It repeats both 0.7 s later, in case the first `STOP` landed inside the node's pre-turn nudge window and was swallowed. This is the most safety-critical part of the hardware integration.

3.6 Summary of Design Decisions

Chapter 4

Implementation

4.1 Speech-to-text Subsystem

4.2 Planner

4.3 Vision-language Verification

4.4 Clarification Manager

4.5 Rover Client and Reactive Layer

4.6 Communication Protocol and Dashboard

4.7 Logging and Measurement Instrumentation

Chapter 5

Experimental Evaluation

This chapter reports what the pipeline did in two kinds of test. The virtual tests drive a simulated rover through a hand-drawn room map using typed commands, and they measure how long a chain of instructions can get before the plan stops matching the command (RQ2). The physical tests run the voice pipeline on the TurboPi in a real room, and they show which command levels in Table 1.1 the pipeline can carry out (RQ1). Every number below comes from the run logs produced by the pipeline itself, and the sample sizes are small enough that I report counts, not percentages.

5.1 Experimental Methodology

5.1.1 Command Corpus Design

Virtual tests. The virtual rover moves through a hand-authored map of one room (`room_tour1`), with every position expressed in a fixed metric frame and drift modelling switched off, so that any failure belongs to the plan and not to the simulated odometry. Commands are typed, and the audio loop is tested only on the physical rover. Four kinds of chain were generated, and each is scored against the sequence of landmarks the command asks for:

- *approach* chains of named landmarks ("go to the couch, then the coffee table, ..."), at depths of 1 to 12 instructions;
- *literal* chains of metric moves and turns ("move forward 0.6 metres, turn right 90 degrees, ..."), at depths of 2 to 8;
- *mixed* chains that combine the two, at depths of 4 to 8;
- *probe* chains of depth 8 that contain one step the room or the rover cannot satisfy: a landmark that is not in the room, or an action outside the capability vocabulary ("pick up the lamp").

Each command was run five times on the local pipeline and three times on the cloud benchmark. A separate context sweep (Section 5.4) used approach chains from depth 4 to 40 in steps of 4, three runs each, at three context settings.

Physical tests. The physical tests use the command levels of Table 1.1, with a level 0 control added in which the rover is only asked to describe what it sees. Each level has a small fixed set of commands built around two objects on the floor of the test room, an orange box and a black bag (later also a blue bottle). Commands are spoken, transcribed by the speech-to-text model, and confirmed by voice before the rover moves. The rover is never given a map or any marker, so every target has to be found by the camera.

5.1.2 Evaluation Metrics

For the virtual tests, a run counts as *correct* when the plan visits exactly the expected landmarks in the expected order (`visits_match`). A run is *completed* when the mission reached the end of the plan, and *halted* when the rover stopped on a blocked step. A *hallucination* is an action, object or location in the plan that is not supported by the command, the room or the rover's capabilities, which is the definition used in Chapter 1. Collisions with virtual obstacles are counted separately.

For the physical tests, each run ends in one of four outcome classes recorded by the pipeline: `completed`, `completed_after_guidance` (the rover stopped, asked what to do next, and finished after the user answered), `halted_after_guidance`, and `aborted`. A run counts as handled when the logged plan matches the command and the rover reached its goal without intervention. I did not score the physical result of every run beyond the log, and Section 5.2 marks where that matters.

5.2 Plan Correctness under Vague Commands

Table 5.1 summarises the final physical batch: 23 runs on the TurboPi with Gemma 4 E4B as both planner and vision model, recorded on 10 and 11 October.

Candidate: L5 has only 2 runs; update this row and the total after the final two L5 runs, and add the raised-target (lamp on the desk) case.

Levels 1 and 2. All six runs produced the expected plan on the first attempt (`turn left, move forward` and `move forward, turn right, move forward`) and finished in 2 to 3 s. These levels need no perception, so with three runs per level they show that the planner maps simple action chains onto the capability vocabulary without help, but they do not give a failure rate.

Level 3. The plan shape was right in every run, because the pipeline turns "drive forward until you see the black bag" into an `approach` step followed by an `observe` and a `report`.

Table 5.1: Physical rover outcomes per command level, final batch (Gemma 4 E4B for planning and vision).

Level	Type	Runs	Done	After guid.	Halted/ aborted
L0	Describe scene (control)	4	4	0	0
L1	Single action	3	3	0	0
L2	Sequence of actions	3	3	0	0
L3	Perception-dependent	4	1	2	1
L4	Conditional	7	6	0	1
L5	Vague	2	2	0	0
	Total	23	19	2	2

Execution was less clean. Two of the four runs needed guidance (one needed three turns), and one halted after guidance. The halted run was the command "please don't do the black bag", which the pipeline resolved to "drive to the black bag" and then drove, so the negation was lost. The same batch contains transcripts that were visibly corrupted by the speech-to-text stage ("arif box" for the orange box, "go towards the orange box if you can see it and right to hear it"), and two of the four level 3 commands were of that kind.

Candidate: Check the spoken wording for these four runs; if "don't" was actually said, report it as a negation failure, otherwise as a transcription error.

Level 4. Five of the seven plans encoded the condition as an explicit `check` step followed by the branch (for example `check, approach, observe, report, stop`), and in those runs the rover either proceeded (three runs) or stayed put (two runs). Two plans dropped the condition entirely: the command was rewritten to "go towards the orange box" and the rover simply went. One of these two, with the black bag as the target, was aborted at an obstacle. Counting a dropped condition as a failure, five of seven level 4 plans were faithful to the command.

Candidate: Add whether each branch decision matched the true scene (box next to bottle or not) for each run; I cannot score this from the logs. Also drop or flag the run where the operator said where the box was.

Level 5. Both final-batch runs completed, but neither is a strong test of vagueness. One names the target outright ("tell me what is near the orange box"), and the other was a garbled command ("go towards the rn fox and then come to see a rally") that the clarification loop turned into "go to the orange box and tell me what you see around it". The clarification step is what made this run work, which is the behaviour the design intends, but a single case cannot support a rate.

Exploratory runs. Earlier runs between 7 and 9 October used different model pairs and are not part of Table 5.1. In six search-and-approach runs with Gemma 4 E4B as the vision model, five halted after guidance with the target not found and one completed. In seven search-and-approach runs with Qwen2.5-VL 3B as the vision model, three completed directly, two completed after guidance, and two aborted. Six further runs on 9 October

(levels 0 to 4, Qwen as the vision model) gave four direct completions, one completion after guidance and one halt after guidance. One of those completions is excluded because the spoken "Thank you." was resolved to the previous command's plan, which shows that the dialogue state carried over between runs. These runs differ in command, planner and scene, so they are not a controlled comparison, and the final batch reached its targets with Gemma as the vision model, so the 8 October failures were not a permanent limit of that model.

Candidate: State what changed between 8 and 10 October (prompt, thresholds, scene), since the vision model alone does not explain the difference. Also say whether the dialogue state carry-over is fixed.

5.3 Executability on the Physical Chassis

Because the rover has no wheel encoders and drives in short stop-look-hop cycles, two measurements decide whether a plan can be carried out safely: how well the ultrasonic sensor stops the rover, and whether the vision model can supply a distance estimate for the hop length.

Ultrasonic stop. The stop threshold was set to 300 mm. Table 5.2 gives the sonar reading after the rover stopped. When the rover approached a box square-on or a wall, the reading after the stop was 184 to 291 mm in all twelve runs, which is up to about 12 cm inside the threshold because the sensor is polled at 100 ms and two consecutive readings are required. The sonar did not see a box approached at 30° to the axis: it read 875 and 1097 mm after the stop, so the beam passed beside the box. A thin chair leg stopped at 426 mm. A low obstacle approached at an angle is therefore the case the ultrasonic layer does not cover, and the vision check is the only protection there.

Table 5.2: Sonar reading after the rover stopped, per case (mm). Threshold 300 mm.

Case	Runs	Sonar after stop (mm)
Box, square-on, from 50 cm	3	203, 231, 211
Box, square-on, from 30 cm	3	239, 291, 237
Box, square-on, from 20 cm	3	184, 203, 200
Wall	3	206, 218, 206
Chair leg	1	426
Box at 30° off-axis	2	1097, 875

Candidate: The sonar bench CSV stores the tape-measured gap in a column named gap_cm, but the values (180 to 291) match the sonar readings in millimetres, so the recorded pass flags (15 of 15 against a 25 cm limit) cannot be trusted. Confirm the units you typed, and recompute the pass count before citing it.

Distance from vision. Section 5.2 relies on the vision model locating the target. I also tested whether it can report how far away the target is, using five true distances from 30 to 150 cm with three frames each. The target was not visible in any frame at 30 cm for the Gemma model and in all three for Qwen in numeric mode, because the camera is too low to see an object that close. For the other distances, Table 5.3 shows that neither

model gave a usable absolute distance. Qwen2.5-VL 3B returned only two values (0.5 m and 1.5 m) in numeric mode, and in bucket mode it returned the same "more than 2 m" bucket for every visible frame. Gemma 4 E4B tracked the ordering of the distances in numeric mode but overestimated by 1.25 m on average, and in bucket mode its ranking was uncorrelated with the true distance. The variable hop length based on these estimates was therefore not adopted, and every mission in the final batch used the fixed hop.

Table 5.3: Vision-model distance estimates against measured distance (visible frames only). MAE is the mean absolute error; the rank correlation is not defined for Qwen in bucket mode because its output was constant.

Model	Mode	Frames	MAE (cm)	Rank corr.
Gemma 4 E4B	numeric	12	125	0.77
Gemma 4 E4B	bucket	11	49	−0.12
Qwen2.5-VL 3B	numeric	12	40	0.43
Qwen2.5-VL 3B	bucket	13	161	n/a

5.4 Compositional Depth and Hallucination Onset

Table 5.4 gives the virtual results for the local pipeline and the cloud benchmark. Across 105 local and 63 cloud chains of the approach, literal and mixed kinds, every plan visited the expected landmarks in the expected order. No hallucination was recorded in any of these 168 runs, so within the depths tested (up to 12 instructions for named landmarks and 8 for literal and mixed chains) I observed no onset of hallucination.

Where execution failed. The only non-completed non-probe runs are the depth-8 mixed chains, where the plan was correct but the mission halted. The command "... go to the bookshelf, turn left 90 degrees, move forward 0.6 metres, turn around, move forward 0.6 metres, ..." produced a plan that followed the wording exactly, and the literal 0.6 m move after the bookshelf collided with a virtual obstacle in five of five local runs and one of three cloud runs. The planner noted this risk itself ("the 0.6 m straight movement ... may encounter an obstacle because `move` has no obstacle avoidance"). This is a plan that is faithful to the command but unsafe in the room, which is a different failure from hallucination and one that a literal command on a small rover will always allow.

Probe chains. The probes test what the pipeline does when one step cannot be carried out. In all 9 cloud runs and 12 of the 15 local runs, the planner dropped the unresolvable step without saying so and returned a seven-step plan for an eight-step command, so the

Table 5.4: Virtual chains by kind and depth. Each cell gives correct plans and completed missions out of the runs ("Corr., compl."), and the median planner latency in seconds ("Plan").

		Local (Gemma 4 E4B)		Cloud benchmark	
Kind	Depth	Corr., compl.	Plan (s)	Corr., compl.	Plan (s)
approach	1	5/5, 5/5	1.0	3/3, 3/3	1.2
approach	2	5/5, 5/5	1.2	3/3, 3/3	1.8
approach	3	5/5, 5/5	4.0	3/3, 3/3	1.8
approach	4	10/10, 10/10	4.2	6/6, 6/6	1.9
approach	6	10/10, 10/10	2.5	6/6, 6/6	2.3
approach	8	10/10, 10/10	2.7	6/6, 6/6	2.3
approach	10	10/10, 10/10	6.0	6/6, 6/6	2.3
approach	12	10/10, 10/10	4.7	6/6, 6/6	3.1
literal	2	5/5, 5/5	1.4	3/3, 3/3	5.2
literal	4	5/5, 5/5	1.9	3/3, 3/3	8.5
literal	6	5/5, 5/5	3.4	3/3, 3/3	6.1
literal	8	10/10, 10/10	3.1	6/6, 6/6	8.3
mixed	4	5/5, 5/5	2.0	3/3, 3/3	7.1
mixed	6	5/5, 5/5	5.6	3/3, 3/3	8.8
mixed	8	5/5, 0/5	7.5	3/3, 2/3	12.3

mission continued past the missing fridge or the unsupported "pick up the lamp". The remaining three local runs, all on the chain ending in the television, kept the step, and two of them halted on it, which is the intended behaviour. No unresolved target was logged for any of the nine cloud probes or the twelve silent local probes (`probe_logged` false), and one local run was counted as a hallucination. The silent dropping of steps is the most important failure in the virtual tests, because it produces a plan that looks valid.

Context window and the plan ceiling. Figure 5.1 shows the context sweep. With the configured `num_ctx` of 8192 the local planner produced correct plans for chains up to 36 instructions in all three runs, and at 40 instructions it produced a 34-step plan in all three runs, dropping the last six visits. The ceiling is therefore a plan-length limit of the model, and not a context limit, since it appears at 8192 where the plan uses only a fraction of the window.

Table 5.5: Correct plans out of three runs, by context setting and chain depth (Gemma 4 E4B).

`num_ctx`	Depth 4–32	36	40	Empty plans
8192	3/3 at every depth	3/3	0/3 (34 steps)	0
2048	3/3 at every depth	2/3	0/3	1 (depth 40)
1024	3/3 at every depth	2/3	1/3	1 (depth 40)

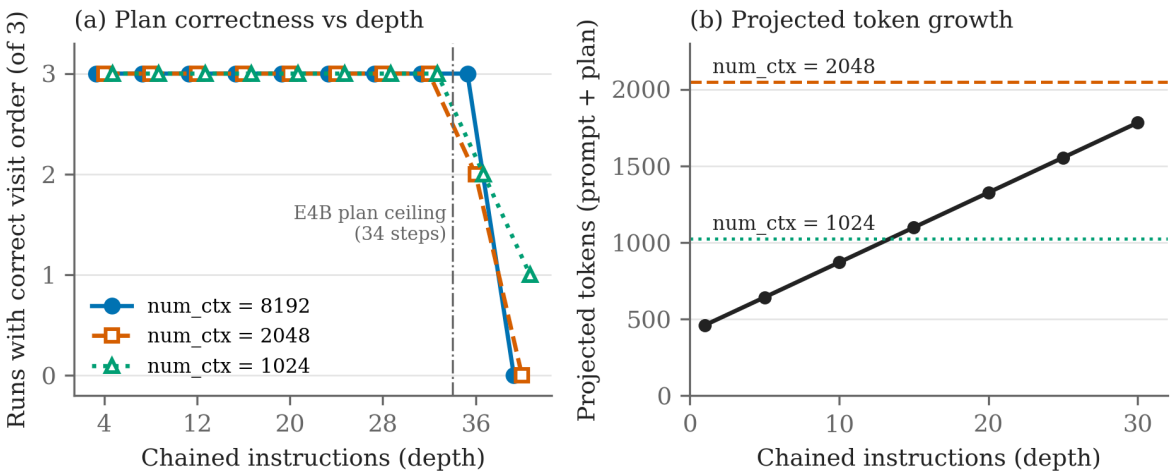


Figure 5.1: (a) Plans with the correct visit order out of three runs, against chain depth, at three context settings (points are offset horizontally for legibility). (b) Projected tokens per planning call against depth, measured to depth 30, with the 1024 and 2048 limits marked.

Measuring the planning prompt gave 460 tokens at depth 1 and 1 782 at depth 30 (about 46 tokens per added instruction), which is 22% of the configured 8192-token window. For a mission that includes replanning with three guidance turns, the largest prompt measured was 1 271 tokens at depth 30, and the largest revision prompt 1 579 tokens. Extrapolating the single-shot slope gives an overflow at about depth 170 for 8192 tokens and about depth 2 800 for the native 128K window, far beyond the 34-step ceiling. The same projection predicts overflow at about depth 13 for 1024 and about depth 36 for 2048 tokens. Table 5.5 does not follow this for 1024: plans stayed correct up to depth 32 at a setting where the projection says the prompt should not fit. The prediction at 2048 matches the observed drop at depth 36 only by coincidence, since the same drop occurs at 8192. One empty plan at depth 40 appeared at each reduced setting and none at 8192, which is suggestive but is one run each.

Candidate: I could not verify that the server enforced the reduced context. Run ollama ps during a sweep (or an overflow probe longer than 1024 tokens) and state the loaded context length here before using the 1024 and 2048 results as evidence either way.

5.5 Reference Comparison against Cloud Models

The cloud benchmark (gpt-5.6-luna through the OpenAI API) matched the local pipeline on plan correctness in every non-probe chain (63 of 63 against 105 of 105), and it behaved the same way on the probes, with the unresolvable step dropped silently in all nine runs. Across the non-probe chains the median planner latency was 3.1 s for the local model and 2.6 s for the cloud model, and the cloud model was slower on literal and mixed chains, where it needed 5 to 12 s against 1.4 to 7.5 s locally (Table 5.4). In these tests the cloud model had no advantage in correctness and none in latency, which is consistent with the project's premise that a small local model is enough for the plan-generation step. Two

limits apply. The cloud runs stop at depth 12 for named landmarks and did not include the depth-40 sweep, so the 34-step ceiling cannot be compared, and no cloud run was made on the physical rover.

5.6 Discussion

RQ1. On the physical rover, the local pipeline handled levels 0 to 2 without help in all ten runs, and levels 3 to 5 ended in a completed mission in 11 of 13 runs (nine directly and two after guidance), with one run halted after guidance at level 3 and one aborted at level 4. Its main weakness at level 4 was dropping the condition in two of seven plans, and its main weakness at level 3 was speech-to-text noise, which twice changed or lost the meaning of the command. Reliability at levels 3 to 5 is therefore supported by a handful of runs and should be reported as a demonstration of capability, not as a rate. The part of the design that earns its place is the guidance loop, which turned several runs that would have ended with "target not found" into completed missions.

RQ2. In the virtual tests no hallucinated step appeared in chains of up to 12 named landmarks or 8 literal instructions, and chains of up to 36 instructions stayed correct at the configured context. The first failure mode is not hallucination but truncation, at about 34 steps, followed by the silent dropping of steps that cannot be carried out. Neither failure is caught by the pipeline at plan time, which argues for a capability check on the plan before departure as the most useful next improvement.

Limitations. The evidence is one room, one rover, and three to seven runs per physical level. Level 5 is not yet adequately covered, the level 4 branches have not been scored against the true scene, and the vision-based distance estimate was not good enough to use. The virtual tests switch odometry drift off and use typed text, so they separate plan quality from perception and speech, and they are not directly comparable to the physical runs. The ultrasonic stop does not protect against low obstacles approached at an angle.

Chapter 6

Conclusion

6.1 Answers to the Research Questions

6.2 Contributions

6.3 Limitations

6.4 Recommendations for Future Work

References

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 287–318, arXiv:2204.01691.

[2] K. Hori, K. Suzuki, and T. Ogata, "Enhancement of long-horizon task planning via active and passive modification in large language models," *Scientific Reports*, vol. 15, p. 7113, 2025.

[3] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, and A. Majumdar, "Robots that ask for help: Uncertainty alignment for large language model planners," in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, arXiv:2307.01928.

[4] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2402.06529.

[5] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, "Real-time anomaly detection and reactive planning with large language models," in *Robotics: Science and Systems (RSS)*, 2024, arXiv:2407.08735.

[6] A. Elhafsi, R. Sinha, C. Agia, E. Schmerling, I. A. D. Nesnas, and M. Pavone, "Semantic anomaly detection with large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 1035–1055, 2023.

[7] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proceedings of*

the IEEE International Conference on Robotics and Automation (ICRA), 2023, pp. 9493–9500.

[8] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, arXiv:2207.05608.

[9] R. Liu, Y. Sun, M. Zhang, H. Bai, X. Yu, T. Yu, C. Yuan, and L. Hou, "Quantization hurts reasoning? an empirical study on quantized reasoning models," 2025.

[10] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. Kåsen, S. Sen, B. A. Mithassel, and A. Soylu, "Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency," *ACM Transactions on Internet of Things*, 2025.

[11] H. Kamper, F. J. Muamba Mukanya, and T. R. Niesler, "Multi-accent acoustic modelling of south african english," *Speech Communication*, vol. 54, no. 6, pp. 801–813, 2012.

[12] C. Jacobs, A. Smith, D. Klop, O. Klejch, F. de Wet, and H. Kamper, "Speech recognition for automatically assessing Afrikaans and isiXhosa preschool oral narratives," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

Appendix A

Appendix title goes here